```
// Project members can view
permit(
    principal,
    action == Action::"doc:view",
    resource in Document::"alpha/*"
)
when { Project::"alpha".members.contains(principal) };

// Project members can edit
permit(
    principal,
    action == Action::"doc:edit",
    resource in Document::"alpha/*"
)
when { Project::"alpha".members.contains(principal) };

// Only employees can share
permit(
    principal in Employee,
    action == Action::"doc:share",
    resource in Document::"alpha/*"
)
when { Project::"alpha".members.contains(principal) };
```

Now the platform can strictly enforce each person's capabilities. All project members, whether employees or customers, can view and edit documents. However, only employees are allowed to share them externally. In practice, ACME still shows these three actions as a bundled `collaborator` role in the UI, but the policies behind them remain precise.

Separating capabilities this way preserves flexibility: customers can continue collaborating on projects while employees are responsible for sharing information outside the project. Fine-grained rules at the policy level provide ACME with strong security assurances, while higher-level roles in the UI keep things simple for daily use.

Figure 10.4 compares ACME’s original bundled policy with the enhanced fine-grained version that separates customer and employee capabilities.